

BEVFormer 學習日報 — 2026年9月28日

學習者：東北大學電子信息專業碩士在讀

項目：BEVFormer 智能駕駛感知模型學習與實戰

日期：2026-09-28

一、今日學習概覽

今日圍繞 BEVFormer 源碼學習 與 工程環境搭建

兩條主線展開，既完成了模型核心機制的理論梳理，也解決了從 GPU 配置到依賴安裝的一系列工程問題，最終成功跑通訓練流程。

核心收穫

模塊	內容	狀態
BEVFormer 理論	BEV Query、空間交叉注意力(SCA)、時間自注意力(TSA)	☐ 已理解
性能優化	融合注意力 + 向量化 SCA	☐ 134x 加速
GPU 配置	RTX 5070 + CUDA 12.8 + PyTorch 2.5	☐ 已啟用
環境管理	Conda 多環境隔離機制	☐ 已掌握
依賴調試	yaml / torch / opencv / tqdm 缺失修復	☐ 已解決

二、BEVFormer 核心機制學習

2.1 BEV Query 生成

BEVFormer 的起點是將 3D 空間離散化為 BEV 網格，每個網格對應一個可學習的 Query：

- 網格配置： bev_h=50, bev_w=50, bev_z=1，對應自車前方 50m × 50m 範圍

- 坐標邊界: `x_bound=[-50, 50]`, `y_bound=[-50, 50]`, `z_bound=[-10, 10]`
- Query 類型: `learnable` (可學習參數), 維度 `[B, H*W*Z, C]`
- 位置編碼: 正弦餘弦位置編碼 (Sinusoidal PE), 為每個 BEV 網格注入空間位置信息

2.2 空間交叉注意力 (Spatial Cross-Attention, SCA)

SCA 是 BEVFormer 的核心創新, 實現從 2D 圖像特徵到 3D BEV 特徵的轉換。

五步流程:

1. 生成 3D Reference Points: 每個 BEV Query 在 3D 空間中對應一個參考點
2. 反歸一化: 將歸一化坐標還原為真實 3D 坐標 (自車坐標系)
3. 多相機投影: 通過外參 (lidar2cam) + 內參 (cam_intrinsic) 將 3D 點投影到各相機 2D 平面
4. 多點偏移採樣: 每個 Query 採樣 `num_points=4` 個偏移點, 獲取圖像特徵
5. 注意力融合: 用 Query 與採樣特徵做注意力加權, 輸出更新後的 BEV 特徵

關鍵優化 (今日完成):

將原本的嵌套循環採樣改為向量化實現:

```
# 向量化多相機採樣 (替代嵌套循環)
v_all = self.v_proj(torch.stack(image_feats, dim=1).flatten(0, 1)) # [B*cam, C, Hf, Wf]
pix_sample = pix_ref.unsqueeze(2).unsqueeze(4) + offsets.unsqueeze(1) # [B, cam, H, N, P, 2]
```

2.3 時間自注意力 (Temporal Self-Attention, TSA)

TSA 通過融合歷史幀的 BEV 特徵, 實現時序信息的積累:

- 歷史幀對齊: 利用 `ego_pose` (自車位姿) 將歷史 BEV 特徵 warp 到當前坐標系
- 自注意力融合: 當前 BEV Query 與對齊後的歷史 BEV 特徵做自注意力
- 緩存機制: 當前幀 BEV 特徵作為下一幀的 `prev_bev` 緩存

2.4 檢測頭與損失函數

檢測頭輸出三個分支: - `cls_logits`: 分類熱圖 (Focal Loss, 處理正負樣本不平衡) -

`center_xy`: 子網格偏移 (L1 Loss, 僅正樣本位置) - `size_whl`: 物體尺寸 (L1 Loss, 僅正樣本位置)

三、性能優化實戰

3.1 問題：訓練極慢 + 顯存爆炸

優化前的訓練狀態： - 速度：46 秒/迭代 - 顯存佔用：17.4 GB - 預計單 epoch：12 小時

3.2 優化方案一：融合注意力

將手動實現的 `matmul + softmax` 替換為 PyTorch 原生融合算子：

```
[ # 優化後：使用 fused attention
out = F.scaled_dot_product_attention(
    q, k, v,
    attn_mask=attn_mask,
    dropout_p=self.dropout.p if self.training else 0.0,
)
```

效果：顯存從 17.4 GB 降至 2.7 GB（6.4 倍降低）

3.3 優化方案二：向量化 SCA

將 SCA 中的多相機、多點採樣從 Python 循環改為張量向量化運算。

效果：訓練速度提升 134 倍，單 epoch 從 12 小時降至 8 分鐘

3.4 優化前後對比

指標	優化前	優化後	提升倍數
單迭代時間	46 s	~0.34 s	134x
顯存佔用	17.4 GB	2.7 GB	6.4x
單 epoch	12 h	8 min	90x

四、GPU 環境配置

4.1 問題：GPU 未被識別

環境顯示 `使用設備：cpu`，但實際擁有 RTX 5070。

4.2 原因

RTX 5070 屬於 Blackwell 架構，需要 CUDA 12.8+ 版本的 PyTorch 才能正確識別。

4.3 解決方案

```
[python -m pip install --upgrade --force-reinstall torch torchvision \
--index-url https://download.pytorch.org/whl/cu128
```

安裝後驗證：

```
[import torch
print(torch.cuda.is_available())      # True
print(torch.cuda.get_device_name(0))  # NVIDIA GeForce RTX 5070
```

五、Python 環境管理學習

5.1 核心問題：為什麼 `import yaml` 失敗？

今日多次遇到 `ModuleNotFoundError`，根本原因是 Conda 多環境隔離。

5.2 環境對比

環境	路徑	已安裝包
base（默認）	C:\Users\PC\miniconda3\	pyyaml, torch(cu128), opencv, tqdm
項目環境	bev\.conda\	初始為空，需逐個安裝

5.3 關鍵規則

解釋器和包必須在同一個環境中才能互相找到。

每個 Conda 環境都是獨立的「房間」，A 環境安裝的包，B 環境無法訪問。

5.4 依賴安裝要點

- PyTorch 需用專用源：`bash python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128`

2. 普通包用 PyPI 源（不可用 `--index-url` 指向 PyTorch 源，否則找不到 opencv 等包）：`bash python -m pip install pyyaml tqdm opencv-python numpy matplotlib scipy`
3. 始終用項目的解釋器安裝：`bash & "c:/Users/PC/Documents/trae_projects/bev/.conda/python.exe" -m pip install <package>`

5.5 最佳實踐

- 每個項目一個獨立 Conda 環境，避免依賴衝突
- 環境搭建完成後導出 `requirements.txt`：`bash python -m pip freeze > requirements.txt`
- 在 VS Code 中鎖定項目解釋器：`Ctrl+Shift+P` → `Python: Select Interpreter`

六、今日踩坑記錄

問題	原因	解決方案
<code>No module named 'yaml'</code>	包裝在 base，運行用項目環境	用項目解釋器裝 pyyaml
<code>No module named 'torch'</code>	項目環境未裝 PyTorch	用 cu128 源安裝
<code>No matching distribution for opencv-python</code>	用了 PyTorch 專用源，沒有 opencv	改用默認 PyPI 源安裝
<code>No module named 'tqdm'</code>	項目環境未裝	安裝 tqdm
GPU 顯示 cpu	PyTorch 版本不支持 Blackwell	安裝 cu128 版本 PyTorch
訓練 46s/iter + 17.4GB	手動注意力 + 循環採樣	融合注意力 + 向量化

七、明日學習計劃

1. 深入 SCA 源碼：理解 3D reference point 反歸一化的數學原理
2. 時間融合細節：分析 ego_pose 對齊與 warp 操作的具體實現
3. 損失函數調試：觀察訓練過程中 loss 曲線變化，調整權重
4. 可視化：使用 visualize.py 查看 BEV 特徵與檢測結果
5. 實車數據遷移：研究如何用真實數據替換 `_generate_ego_poses` 等模擬函數

八、學習心得

1. 理論結合實踐：BEVFormer 的 SCA

機制通過動手實現向量化版本，對投影幾何和注意力機制有了更深的直觀理解。

2. 工程能力提升：從 Conda 環境隔離到 PyTorch CUDA 版本選擇，解決了多個真實工程問題，對 Python 生態的理解更加系統。

3. 性能優化思維：融合注意力和向量化的優化讓我體會到「避免大矩陣實例化」和「用張量運算替代循環」的重要。

4. 持續學習：BEV 只是起點，後續還需關注 VLA、世界模型（WAM）等前沿方向，理解它們的演進關係。